Nile University

Lecture (07)
Pointers & Arrays

Nile University

Lecture (07) part 01
chapter 10 : Pointers

Topics

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship Between Arrays and Pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

Topics (continued)

10.1 Pointers and the Address Operator

- Each variable in a program is stored at a unique location in memory that has an address
- Use the address operator **&** to get the address of a variable:

```
int num = -23;
cout << &num; // prints address
              // in hexadecimal
```

- The address of a memory location is a pointer

10.2 Pointer Variables

- Pointer variable (pointer): a variable that holds an address
- Pointers provide an alternate way to access memory locations

Pointer Variables

- Definition:

```
int  *intptr;
```

- Read as:
  "**`intptr`** can hold the address of an int" or "the variable that **`intptr`** points to has type int"
- The spacing in the definition does not matter:

```
int * intptr;
int*  intptr;
```

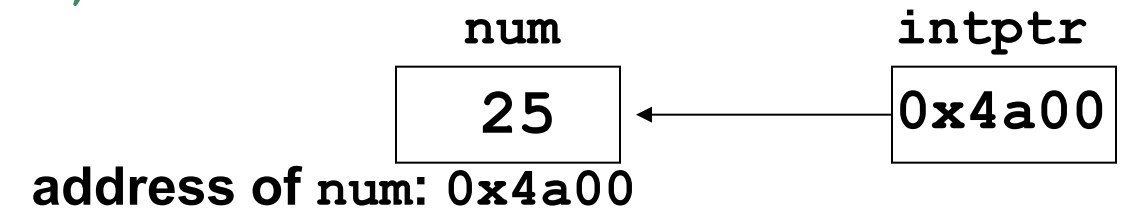
- **`*`** is called the indirection operator

Pointer Variables

- Assignment:

```
int num = 25;
int *intptr;
intptr = &num;
```

- Memory layout:



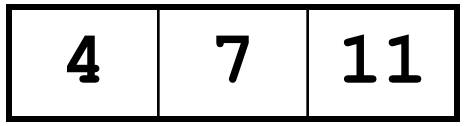
- Can access **num** using **intptr** and indirection operator *****:

```
cout << intptr;  // prints 0x4a00
cout << *intptr; // prints 25
*intptr = 20;    // puts 20 in num
```

10.3 The Relationship Between Arrays and Pointers

An array name is the starting address of the array

```
int vals[] = {4, 7, 11};
```




```
cout << vals;          // displays 0x4a00
cout << vals[0]; // displays 4
```

starting address of `vals`: `0x4a00`

The Relationship Between Arrays and Pointers

- An array name can be used as a pointer constant

```
int vals[] = {4, 7, 11};
cout << *vals;    // displays 4
```

- A pointer can be used as an array name

```
int *valptr = vals;
cout << valptr[1]; // displays 7
```

Pointers in Expressions

- Given:

```
int vals[]={4,7,11};
int *valptr = vals;
```

- What is **`valptr + 1`**?
- It means (address in **`valptr`**) + (1 * size of an **`int`**)

```
cout << *(valptr+1); // displays 7
cout << *(valptr+2); // displays 11
```

- Must use **`( )`** in expression

Array Access

Array elements can be accessed in many ways

Array access method	Example
array name and `[ ]`	`vals[2] = 17;`
pointer to array and `[ ]`	`valptr[2] = 17;`
array name and subscript arithmetic	`*(vals+2) = 17;`
pointer to array and subscript arithmetic	`*(valptr+2) = 17;`

Array Access

- Array notation

```
vals[i]
```

  is equivalent to the pointer notation

```
*(vals + i)
```

- No bounds checking is performed on array access

10.4 Pointer Arithmetic

Some arithmetic operators can be used with pointers:

- Increment and decrement operators **++**, **--**
- Integers can be added to or subtracted from pointers using the operators **+**, **-**, **+=**, and **-=**
- One pointer can be subtracted from another by using the subtraction operator **-**

Pointer Arithmetic

Assume the variable definitions

```
int vals[]={4,7,11};
int *valptr = vals;
```

Examples of use of **++** and **--**

```
valptr++; // points at 7
valptr--; // now points at 4
```

More on Pointer Arithmetic

Assume the variable definitions:

```
int vals[]={4,7,11};
int *valptr = vals;
```

Example of the use of + to add an int to a pointer:

```
cout << *(valptr + 2)
```

This statement will print 11

More on Pointer Arithmetic

Assume the variable definitions:

```
int vals[]={4,7,11};
int *valptr = vals;
```

Example of use of +=:

```
valptr = vals; // points at 4
valptr += 2;   // points at 11
```

More on Pointer Arithmetic

Assume the variable definitions

```
int vals[] = {4,7,11};
int *valptr = vals;
```

Example of pointer subtraction

```
valptr += 2;
cout << valptr - val;
```

This statement prints **2**: the number of **ints** between **valptr** and **val**

10.5 Initializing Pointers

- Can initialize to NULL or 0 (zero)

```
int *ptr = NULL;
```

- Can initialize to addresses of other variables

```
int num, *numPtr = &num;
int val[ISIZE], *valptr = val;
```

- Initial value must have correct type

```
float cost;
int *ptr = &cost; // won't
work
```

10.6 Comparing Pointers

- Relational operators can be used to compare addresses in pointers
- Comparing addresses in pointers is not the same as comparing contents pointed at by pointers:

```
if (ptr1 == ptr2)    // compares
                     // addresses
if (*ptr1 == *ptr2) // compares
                       // contents
```

10.7 Pointers as Function Parameters

- A pointer can be a parameter
- It works like a reference parameter to allow changes to argument from within a function
- A pointer parameter must be explicitly dereferenced to access the contents at that address

Pointers as Function Parameters

Requires:

1) asterisk * on parameter in prototype and heading

```
void getNum(int *ptr);
```

2) asterisk * in body to dereference the pointer

```
cin >> *ptr;
```

3) address as argument to the function in the call

```
getNum(&num);
```

Pointers as Function Parameters

```
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
int num1 = 2, num2 = -3;
swap(&num1, &num2);  //call
```

10.8 Ponters to Constants and Constant Pointers

- Pointer to a constant: cannot change the value that is pointed at

- Constant pointer: the address in the pointer cannot change after the pointer is initialized

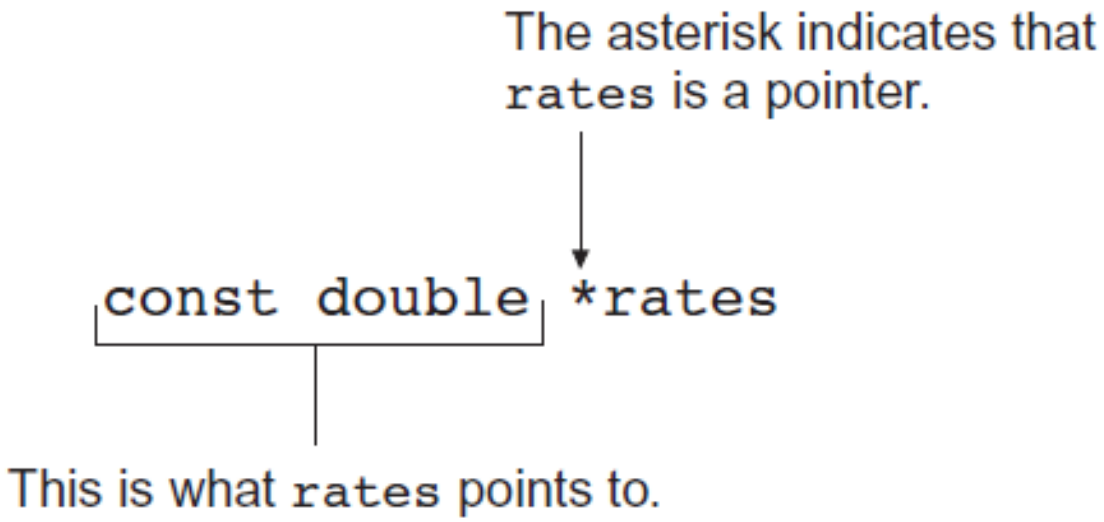
Ponters to Constant

- Must use **`const`** keyword in pointer definition:

```
const double taxRates[] =
                    {0.65, 0.8, 0.75};
const double *ratePtr;
```

- Use **`const`** keyword for pointers in function headers to protect data from modification from within function

Pointer to Constant – What does the Definition Mean?



Read as: “rates is a pointer to a constant that is a double.”

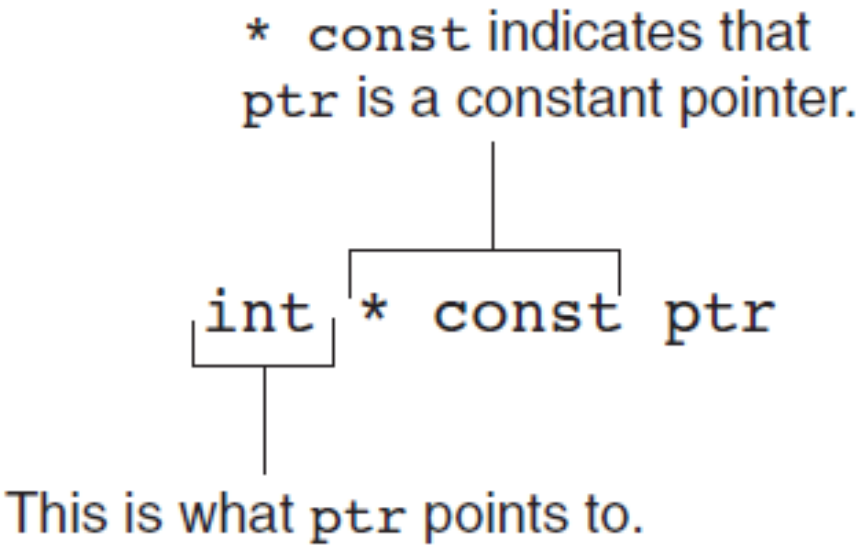
Constant Pointers

- Defined with **`const`** keyword adjacent to variable name:

```
int classSize = 24;
int * const classPtr = &classSize;
```

- Must be initialized when defined
- Can be used without initialization as a function parameter
  - Initialized by argument when function is called
  - Function can receive different arguments on different calls
- While the address in the pointer cannot change, the data at that address may be changed

Constant Pointer – What does the Definition Mean?



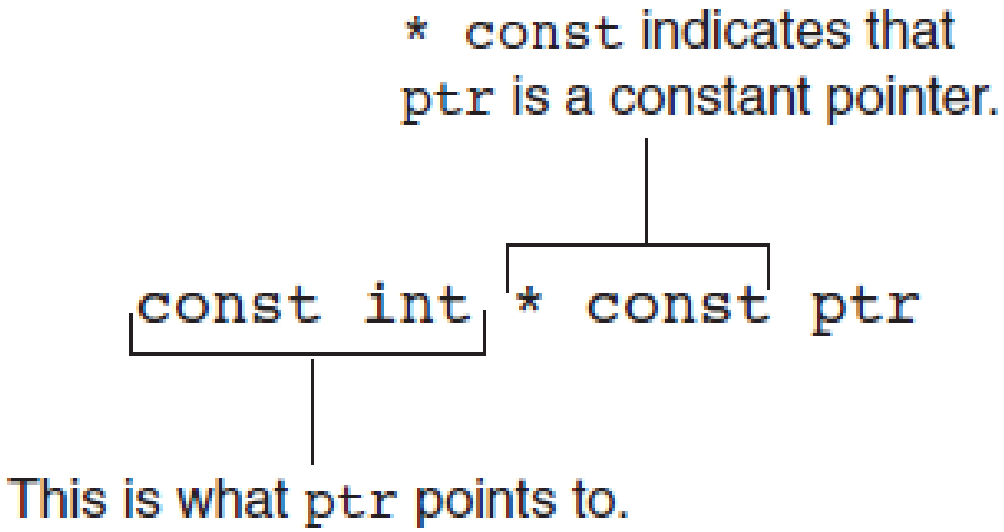
Read as: “pts is a constant pointer to an int.”

Constant Pointer to Constant

- Can combine pointer to constants and constant pointers:

```
int size = 10;
const int * const ptr = &size;
```

- What does it mean?



10.9 Dynamic Memory Allocation

- Can allocate storage for a variable while program is running
- Uses **`new`** operator to allocate memory

```
double *dptr;
dptr = new double;
```

- **`new`** returns address of memory location

Dynamic Memory Allocation

- Can also use **`new`** to allocate array

```
arrayPtr = new double[25];
```

  - Program may terminate if there is not sufficient memory

- Can then use **`[ ]`** or pointer arithmetic to access array

Dynamic Memory Example

```
int *count, *arrayptr;
count = new int;
cout <<"How many students? ";
cin >> *count;
arrayptr = new int[*count];

for (int i=0; i<*count; i++)
{
  cout << "Enter score " << i << ": ";
  cin >> arrayptr[i];
}
```

Releasing Dynamic Memory

- Use **`delete`** to free dynamic memory

```
delete count;
```

- Use **`delete []`** to free dynamic array memory

```
delete [] arrayptr;
```

- Only use **`delete`** with dynamic memory!

Dangling Pointers and Memory Leaks

- A pointer is dangling if it contains the address of memory that has been freed by a call to **`delete`**.
  - Solution: set such pointers to 0 as soon as memory is freed.
- A memory leak occurs if no-longer-needed dynamic memory is not freed. The memory is unavailable for reuse within the program.
  - Solution: free up dynamic memory after use

10.10 Returning Pointers from Functions

- Pointer can be return type of function

```
int* newNum();
```

- The function must not return a pointer to a local variable in the function
- The function should only return a pointer
  - to data that was passed to the function as an argument
  - to dynamically allocated memory

10.11 Pointers to Class Objects and Structures

- Can create pointers to objects and structure variables

```
struct Student {…};
class Square {…};
Student stu1;
Student *stuPtr = &stu1;
Square sq1[4];
Square *squarePtr = &sq1[0];
```

- Need to use **()** when using ***** and **.** operators

```
(*stuPtr).studentID = 12204;
```

Structure Pointer Operator

- Simpler notation than **`(*ptr).member`**
- Use the form **`ptr->member:`**

```
stuPtr->studentID = 12204;
squarePtr->setSide(14);
```

in place of the form **`(*ptr).member:`**

```
(*stuPtr).studentID = 12204;
(*squarePtr).setSide(14);
```

Dynamic Memory with Objects

- Can allocate dynamic structure variables and objects using pointers:

```
stuPtr = new Student;
```

- Can pass values to constructor:

```
squarePtr = new Square(17);
```

- **`delete`** causes destructor to be invoked:

```
delete squarePtr;
```

Structure/Object Pointers as Function Parameters

- Pointers to structures or objects can be passed as parameters to functions
- Such pointers provide a pass-by-reference parameter mechanism
- Pointers must be dereferenced in the function to access the member fields

Controlling Memory Leaks

- Memory that is allocated with **`new`** should be deallocated with a call to **`delete`** as soon as the memory is no longer needed. This is best done in the same function as the one that allocated the memory.
- For dynamically-created objects, **`new`** should be used in the constructor and **`delete`** should be used in the destructor

10.12 Selecting Members of Objects

Situation: A structure/object contains a pointer as a member. There is also a pointer to the structure/object.

Problem: How do we access the pointer member via the structure/object pointer?

```
struct GradeList
  { string courseNum;
    int * grades;
  }
GradeList test1, *testPtr = &test1;
```

Selecting Members of Objects

Expression	Meaning
`testPtr->grades`	Access the grades pointer in `test1`. This is the same as `(*testPtr).grades`
`*testPtr->grades`	Access the value pointed at by `testPtr->grades`. This is the same as `*(*testPtr).grades`
`*test1.grades`	Access the value pointed at by `test1.grades`

Nile University

Lecture (07) part 02
chapter 8 : Arrays

Topics

8.1 Arrays Hold Multiple Values

8.2 Accessing Array Elements

8.3 Inputting and Displaying Array Contents

8.4 Array Initialization

8.5 Processing Array Contents

8.6 Using Parallel Arrays

Topics (continued)

8.7 The **`typedef`** Statement

8.8 Arrays as Function Arguments

8.9 Two-Dimensional Arrays

8.10 Arrays with Three or More Dimensions

8.11 Vectors

8.12 Arrays of Objects

8.1 Arrays Hold Multiple Values

- Array: variable that can store multiple values of the same type
- Values are stored in consecutive memory locations
- Declared using `[]` operator

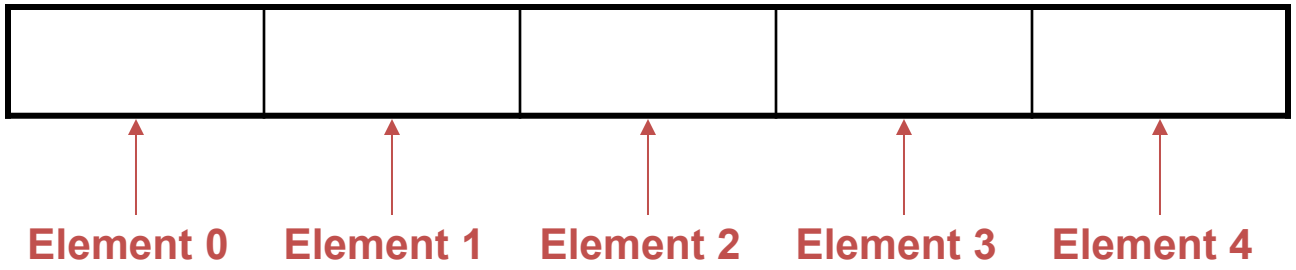
```
const int ISIZE = 5;
int tests[ISIZE];
```

Array Storage in Memory

The definition

```
int tests[ISIZE];  // ISIZE is 5
```

allocates the following memory



Array Terminology

In the definition `int tests[ISIZE];`

- **`int`** is the data type of the array elements
- **`tests`** is the name of the array
- **`ISIZE`**, in **`[ISIZE]`**, is the size declarator. It shows the number of elements in the array.
- The size of an array is the number of bytes allocated for it

*(number of elements) * (bytes needed for each element)*

Array Terminology Examples

Examples:

Assumes **`int`** uses 4 bytes and **`double`** uses 8 bytes

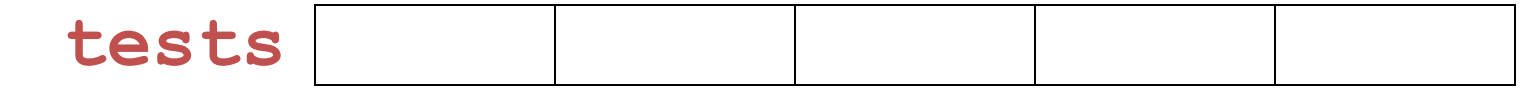
```
const int ISIZE = 5, DSIZE = 10;
int tests[ISIZE]; // holds 5 ints, array
                  // occupies 20 bytes
double volumes[DSIZE];// holds 10
doubles,
                              // array occupies
                              // 80 bytes
```

8.2 Accessing Array Elements

- Each array element has a subscript, used to access the element.
- Subscripts start at 0

Accessing Array Elements

Array elements (accessed by array name and subscript) can be used as regular variables




```
tests[0] = 79;
cout << tests[0];
cin  >> tests[1];
tests[4] = tests[0] + tests[1];
cout << tests; // illegal due to
               // missing subscript
```

8.3 Inputting and Displaying Array Contents

`cout` and **`cin`** can be used to display values from and store values into an array

```
const int ISIZE = 5;

int tests[ISIZE]; // Define 5-elt. array
cout << "Enter first test score ";
cin  >>  tests[0];
```

Array Subscripts

- Array subscript can be an integer constant, integer variable, or integer expression
- Examples:

	Subscript is
`cin >> tests[3];`	int constant
`cout << tests[i];`	int variable
`cout << tests[i+j];`	int expression

Accessing All Array Elements

To access each element of an array

- Use a loop
- Let the loop control variable be the array subscript
- A different array element will be referenced each time through the loop

```
for (i = 0; i < 5; i++)
    cout << tests[i] << endl;
```

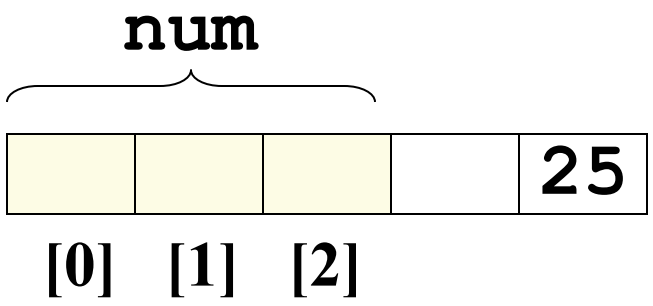
Getting Array Data from a File

```
const int ISIZE = 5, sales[ISIZE];
ifstream dataFile;
datafile.open("sales.dat");
if (!dataFile)
   cout << "Error opening data file\n";
else
{  // Input daily sales
   for (int day = 0; day < ISIZE; day++)
      dataFile >> sales[day];
   dataFile.close();
}
```

No Bounds Checking

- There are no checks in C++ that an array subscript is in range
- An invalid array subscript can cause program to overwrite other memory
- Example:

```
const int ISIZE = 3;
int i = 4;
int num[ISIZE];
num[i] = 25;
```



Off-By-One Errors

- Most often occur when a program accesses data one position beyond the end of an array, or misses the first or last element of an array.
- Don’t confuse the ordinal number of an array element (first, second, third) with its subscript (0, 1, 2)

8.4 Array Initialization

- Can be initialized during program execution with assignment statements

```
tests[0] = 79;
tests[1] = 82; // etc.
```

- Can be initialized at array definition with an initialization list

```
const int ISIZE = 5;
int tests[ISIZE] = {79,82,91,77,84};
```

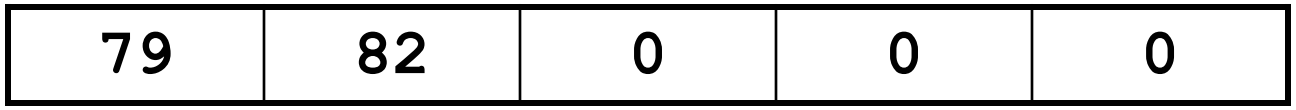
Start at element 0 or 1?

- You may choose to declare arrays to be one larger than needed. This allows you to use the element with subscript 1 as the ‘first’ element, etc., and may minimize off-by-one errors.
- The element with subscript 0 is not used.
- This is most often done when working with ordered data, *e.g.*, months of the year or days of the week

Partial Array Initialization

- If array is initialized at definition with fewer values than the size declarator of the array, remaining elements will be set to **0** or the empty string

```
int tests[ISIZE] = {79, 82};
```

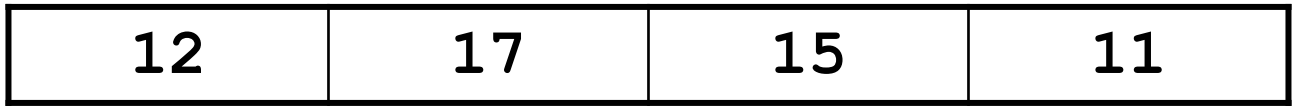


- Initial values used in order; cannot skip over elements to initialize noncontiguous range
- Cannot have more values in initialization list than the declared size of the array

Implicit Array Sizing

- Can determine array size by the size of the initialization list

```
short quizzes[]={12,17,15,11};
```



- Must use either array size declarator or initialization list when array is defined

8.5 Processing Array Contents

- Array elements can be
  - treated as ordinary variables of the same type as the array
  - used in arithmetic operations, in relational expressions, etc.
- Example:

```
if (principalAmt[3] >= 10000)
  interest = principalAmt[3] * intRate1;
else
  interest = principalAmt[3] * intRate2;
```

Using Increment and Decrement Operators with Array Elements

When using **++** and **--** operators, don't confuse the element with the subscript

```
tests[i]++;  // adds 1 to tests[i]
tests[i++];  // increments i, but has
             // no effect on tests
```

Copying One Array to Another

- Cannot copy with an assignment statement:

```
tests2 = tests;  //won't work
```

- Must instead use a loop to copy element-by-element:

```
for (int indx=0; indx < ISIZE; indx++)
   tests2[indx] = tests[indx];
```

Are Two Arrays Equal?

- Like copying, cannot compare in a single expression:

```
if (tests2 == tests)
```

- Use a while loop with a boolean variable:

```
bool areEqual=true;
int indx=0;
while (areEqual && indx < ISIZE)
{
   if(tests[indx] != tests2[indx]
      areEqual = false;
}
```

Sum, Average of Array Elements

- Use a simple loop to add together array elements

```
float average, sum = 0;
for (int tnum=0; tnum< ISIZE; tnum++)
    sum += tests[tnum];
```

- Once summed, average can be computed

```
average = sum/ISIZE;
```

Largest Array Element

- Use a loop to examine each element and find the largest element (*i.e.,* one with the largest value)

```
int largest = tests[0];
for (int tnum = 1; tnum < ISIZE;
 tnum++)
{  if (tests[tnum] > largest)
      largest = tests[tnum];
}
cout << "Highest score is " << largest;
```

- A similar algorithm exists to find the smallest element

Partially-Filled Arrays

- The exact amount of data (and, therefore, array size) may not be known when a program is written.
- Programmer makes best estimate for maximum amount of data, sizes arrays accordingly. A sentinel value can be used to indicate end-of-data.
- Programmer must also keep track of how many array elements are actually used

Using Arrays vs. Using Simple Variables

- An array is probably not needed if the input data is only processed once:
  - Find the sum or average of a set of numbers
  - Find the largest or smallest of a set of values
- If the input data must be processed more than once, an array is probably a good idea:
  - Calculate the average, then determine and display which values are above the average and which are below the average

C-Strings and `string` Objects

Can be processed using array name

- Entire string at once, or
- One element at a time by using a subscript

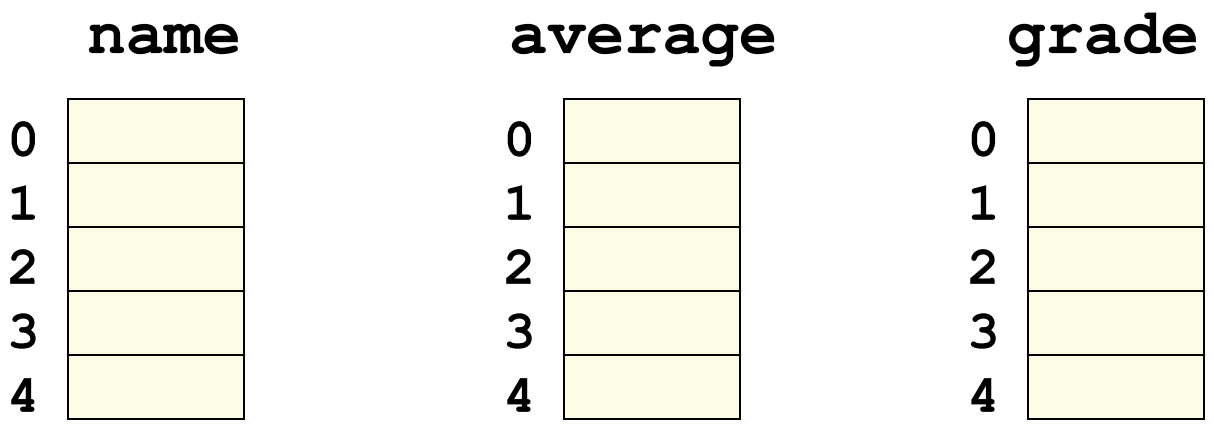
```
string city;
cout << "Enter city name: ";
cin  >> city;
```

8.6 Using Parallel Arrays

- Parallel arrays: two or more arrays that contain related data
- Subscript is used to relate arrays
  - elements at same subscript are related
- The arrays do not have to hold data of the same type

Parallel Array Example

```
const int ISIZE = 5;
string name[ISIZE];   // student name
float average[ISIZE]; // course average
char grade[ISIZE];    // course grade
```



Parallel Array Processing

```
const int ISIZE = 5;
string name[ISIZE];   // student name
float average[ISIZE]; // course average
char grade[ISIZE];    // course grade
...
for (int i = 0; i < ISIZE; i++)
    cout << " Student: " << name[i]
         << " Average: " << average[i]
         << " Grade: "   << grade[i]
         << endl;
```

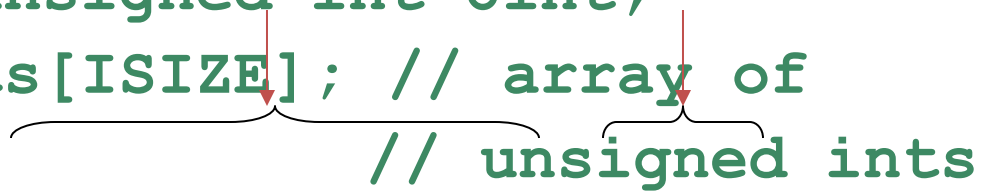
8.7 The `typedef` Statement

- Creates an alias for a simple or structured data type
- Format:

```
typedef existingType newName;
```

- Example:

```
typedef unsigned int Uint;
Uint tests[ISIZE]; // array of
                   // unsigned ints
```



Uses of `typedef`

- Used to make code more readable
- Can be used to create alias for an array of a particular type

```
// Define yearArray as a data type
// that is an array of 12 ints
typedef int yearArray[MONTHS];

// Create two of these arrays
yearArray highTemps, lowTemps;
```

8.8 Arrays as Function Arguments

- Passing a single array element to a function is no different than passing a regular variable of that data type
- Function does not need to know that the value it receives is coming from an array

```
displayValue(score[i]);      // call

void displayValue(int item) // header
{  cout << item << endl;
}
```

Passing an Entire Array

- To define a function that has an array parameter, use empty `[]` to indicate the array argument
- To pass an array to a function, just use the array name

```
// Function prototype
void showScores(int []);

// Function header
void showScores(int tests[])

// Function call
showScores(tests);
```

Passing an Entire Array

- Use the array name, without any brackets, as the argument
- Can also pass the array size so the function knows how many elements to process

```
showScores(tests, 5);          // call
void showScores(int[],int); // prototype
void showScores(int A[],
                int size) // header
```

Using `typedef` with a Passed Array

Can use **`typedef`** to simplify function prototype and heading

```
// Make intArray an integer array
// of unspecified size
typedef int intArray[];

// Function prototype
void showScores(intArray, int);

// Function header
void showScores(intArray tests,
                int size)
```

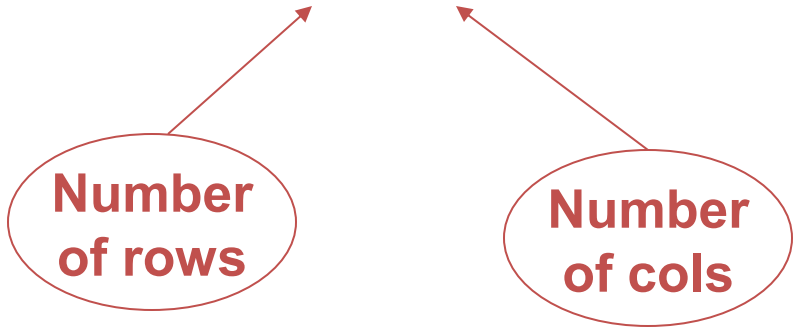
Modifying Arrays in Functions

- Array parameters in functions are similar to reference variables
- Changes made to array in a function are made to the actual array in the calling function
- Must be careful that an array is not inadvertently changed by a function
- Can use const keyword in prototype and header to prevent changes

8.9 Two-Dimensional Arrays

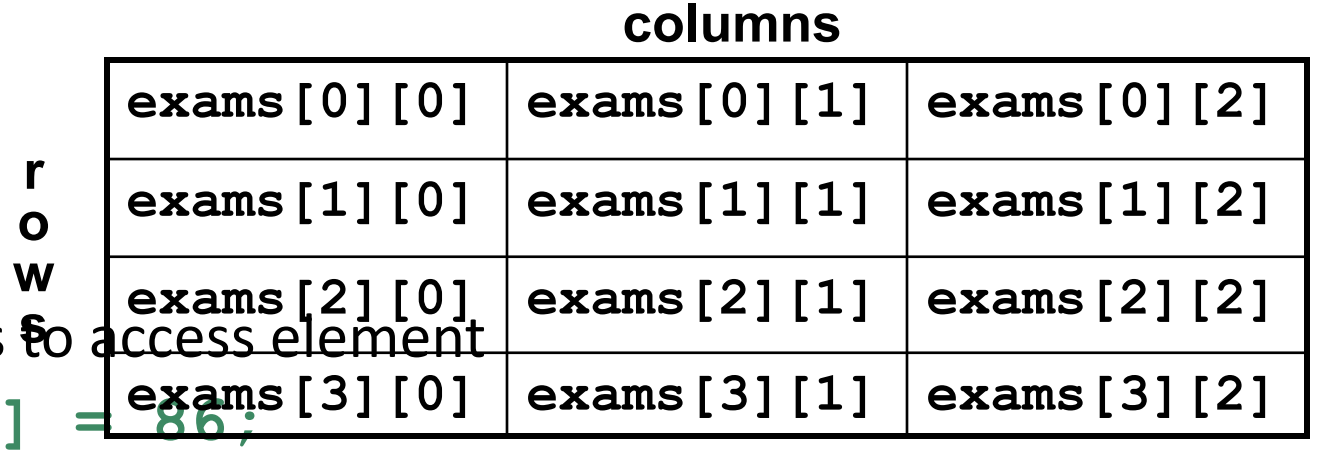
- Can define one array for multiple sets of data
- Like a table in a spreadsheet
- Use two size declarators in definition

```
int exams[4][3];
```



Two-Dimensional Array Representation

```
int exams[4][3];
```



Use two subscripts to access element

```
exams[2][2] = 86;
```

Initialization at Definition

- Two-dimensional arrays are initialized row-by-row

```
int exams[2][2] = { {84, 78},
                    {92, 97} };
```

- Can omit inner `{ }`

84	78
92	97

Passing a Two-Dimensional Array to a Function

- Use array name and number of columns as arguments in function call

```
getExams(exams, 2);
```

- Use empty `[]` for row and a size declarator for col in the prototype and header

```
// Prototype, where NUM_COLS is 2
void getExams(int[][NUM_COLS], int);

// Header
void getExams
          (int exams[][NUM_COLS], int rows)
```

Using `typedef` with a Two-Dimensional Array

Can use `typedef` for simpler notation

```
typedef int intExams[][2];
   ...
// Function prototype
void getExams(intExams, int);
// Function header
void getExams(intExams exams, int rows)
```

2D Array Traversal

- Use nested loops, one for row and one for column, to visit each array element.
- Accumulators can be used to sum the elements row-by-row, column-by-column, or over the entire array.

8.10 Arrays with Three or More Dimensions

- Can define arrays with any number of dimensions

```
short rectSolid(2,3,5);
double timeGrid(3,4,3,4);
```

- When used as parameter, specify size of all but 1st dimension

```
void getRectSolid(short [][3][5]);
```

8.11 Vectors

- Holds a set of elements, like an array
- Flexible number of elements - can grow and shrink
  - No need to specify size when defined
  - Automatically adds more space as needed
- Defined in the Standard Template Library (STL)
  - Covered in a later chapter
- Must include **`vector`** header file to use vectors

```
#include <vector>
```

Vectors

- Can hold values of any type
  - Type is specified when a vector is defined

```
vector<int> scores;
vector<double> volumes;
```

- Can use `[]` to access elements

Defining Vectors

- Define a vector of integers (starts with 0 elements)

```
vector<int> scores;
```

- Define **`int`** vector with initial size 30 elements

```
vector<int> scores(30);
```

- Define 20-element **`int`** vector and initialize all elements to 0

```
vector<int> scores(20, 0);
```

- Define **`int`** vector initialized to size and contents of vector **`finals`**

```
vector<int> scores(finals);
```

Growing a Vector's Size

- Use **push_back** member function to add an element to a full array or to an array that had no defined size

```
// Add a new element holding a 75
scores.push_back(75);
```

- Use **size** member function to determine number of elements currently in a vector

```
howbig = scores.size();
```

Removing Vector Elements

- Use `pop_back` member function to remove last element from vector

```
scores.pop_back();
```

- To remove all contents of vector, use `clear` member function

```
scores.clear();
```

- To determine if vector is empty, use `empty` member function

```
while (!scores.empty()) ...
```

8.14 Arrays of Objects

- Objects can also be used as array elements

```
class Square
{ private:
    int side;
  public:
    Square(int s = 1)
    { side = s; }
    int getSide()
    { return side; }
};
Square shapes[10];  // Create array of 10
                    // Square objects
```

Arrays of Objects

- Like an array of structures, use an array subscript to access a specific object in the array
- Then use dot operator to access member methods of that object

```
for (i = 0; i < 10; i++)
   cout << shapes[i].getSide() << endl;
```

Initializing Arrays of Objects

- Can use default constructor to perform same initialization for all objects
- Can use initialization list to supply specific initial values for each object

```
Square shapes[5] = {1,2,3,4,5};
```

- Default constructor is used for the remaining objects if initialization list is too short

```
Square boxes[5] = {1,2,3};
```

Initializing Arrays of Objects

If an object is initialized with a constructor that takes > 1 argument, the initialization list must include a call to the constructor for that object

```
Rectangle spaces[3] =
{ Rectangle(2,5),
  Rectangle(1,3),
  Rectangle(7,7)  };
```

Arrays of Structures

- Structures can be used as array elements

```
struct Student
{
   int studentID;
   string name;
   short year;
   double gpa;
};
const int CSIZE = 30;
Student class[CSIZE]; // Holds 30
                      // Student structures
```

Arrays of Structures

- Use array subscript to access a specific structure in the array
- Then use dot operator to access members of that structure

```
cin  >> class[25].studentID;

cout << class[i].name << " has GPA "
     << class[i].gpa << endl;
```

Thanks,..

See you next week isA,..